

Instituto Federal de Educação, Ciência e Tecnologia
Campus Inhumas

Algoritmos e Técnicas de Programação II

Structs e estruturas de dados heterogêneas

Prof. Dr. Victor Hugo Lázaro Lopes

AGENDA

- Estruturas de dados heterogêneas
- Exercícios

Novos tipos de dados?

Você já se deparou com um problema de programação no qual deseje agrupar sob um único nome um conjunto de tipos de dados não similares?

Utilizaria matrizes?

O problema de agrupar dados desiguais em C é resolvido pelo uso de **estruturas** (structs).

- ▶ *Estruturas* são tipos de variáveis que agrupam dados geralmente desiguais; ao passo que matrizes agrupam dados similares
 - ▶ Os itens de dados da estrutura são chamados de **membros**, e os da matriz, de **elementos**
 - ▶ Em algumas linguagens de programação, *estruturas* são chamadas de *registros*

Novos tipos de dados?

O exemplo tradicional de uma estrutura é o registro de uma folha de pagamento.

Um funcionário é descrito por um conjunto de atributos, tais como nome (caractere), número de seu departamento (inteiro), salário (real).

Possivelmente, haverão outros funcionários, e você vai querer que seu programa os guarde, formando uma matriz de estruturas.

- ▶ Você já conhece os cinco tipos de dados simples que estão pré-definidos no compilador: **char**, **int**, **float**, **double** e **void**
 - ▶ Com base nesses tipos, podemos definir tipos complexos, que possibilitem agrupar um conjunto de variáveis de tipos diferentes sob um único nome

Novos tipos de dados?

Em C, por meio da **palavra-chave struct**, definimos um novo tipo de dado.

- ☐ Definir um novo tipo de dado significa informar ao compilador
 - nome
 - tamanho em bytes
 - forma como deve ser armazenado e recuperado na memória

Após ter sido definido, o novo tipo existe e pode ser utilizado para criar variáveis de modo similar a qualquer tipo simples.


```
exemplos_aulas > structs > C structsA.c > main()
1  #include <stdio.h>
2  #include <string.h>
3
4  struct Aluno{
5      char nome[50];
6      int matricula;
7  };
8
9  int main(){
10     struct Aluno aluno1;
11     strcpy(aluno1.nome, "José da Silva");
12     aluno1.matricula = 2024001;
13
14     printf("Dados do aluno:\n");
15     printf("Nome: %s\n", aluno1.nome);
16     printf("Matrícula: %d\n", aluno1.matricula);
17
18     return 0;
19 }
```

```
exemplos_aulas > structs > C main()
1  #include <stdio.h>
2  #include <string.h>
3
4  struct Aluno{
5      char nome[50];
6      int matricula;
7  };
8
9  int main(){
10     struct Aluno aluno1;
11     strcpy(aluno1.nome, "José da Silva");
12     aluno1.matricula = 2024001;
13
14     printf("Dados do aluno:\n");
15     printf("Nome: %s\n", aluno1.nome);
16     printf("Matrícula: %d\n", aluno1.matricula);
17
18     return 0;
19 }
```

Palavra-chave

Etiqueta

Membros

Atenção!!!

exemplos_aulas > structs > C structsA.c > main()

```
1  #include <stdio.h>
```

A definição de uma estrutura não cria variável alguma, somente informa ao compilador as características de um novo tipo de dado

```
5      char nome[50];  
6      int matricula;  
7  };
```

```
8  
9  int main(){  
10     struct Aluno aluno1;  
11     strcpy(aluno1.nome, "José da Silva");  
12     aluno1.matricula = 2024001;  
13  
14     printf("Dados do aluno:\n");  
15     printf("Nome: %s\n", aluno1.nome);  
16     printf("Matrícula: %d\n", aluno1.matricula);  
17  
18     return 0;  
19 }
```

Definindo uma variável do tipo Aluno

Operador "ponto": acesso aos membros da estrutura

Acessando os membros da estrutura

Há uma outra maneira de criar a estrutura e a variável:

```
struct Aluno          /*Inicio da definição da estrutura */  
{  
    int nmat;          /* Número da matrícula */  
    float nota[3];     /* Notas */  
    float media;       /* Média */  
} Jose;               /* Cria a variável Jose do tipo Aluno*/
```

```
struct Aluno          /*Inicio da definição da estrutura */  
{  
    int nmat;          /* Número da matrícula */  
    float nota[3];     /* Notas */  
    float media;       /* Média */  
} Jose, Ana, Joao;    /* Cria a variáveis do tipo Aluno*/
```

Outra sintaxe

Usa a palavra-chave typedef, que cria “apelidos” para tipos existentes.

exemplos_aulas > structs > C structsC.c > main()

```
1  #include <stdio.h>
2  #include <string.h>
3
4  typedef struct{
5      int pecas;
6      float preco;
7  } Venda;
8
9  int main(){
10     Venda A = {20, 110.0}, B = {3, 16.5}, Total;
11
12     return 0;
13 }
```

A etiqueta é suprimida, e o tipo é definido como “apelido” ao fim da estrutura

Assim o tipo Venda pode ser utilizado diretamente

Novos nomes para os tipos existentes: typedef

- ▶ Declarações com **typedef** não produzem novos tipos de dados, apenas criam novos nomes (sinônimos) para os tipos existentes

- ▶ **Sintaxe**

```
typedef tipo-existente sinônimo;
```

- ▶ Observe o fragmento de código abaixo

```
typedef unsigned char BYTE; /* Cria o sinônimo BYTE */
typedef unsigned int uint; /* Cria o sinônimo uint */

int main()
{
    BYTE ch; /* Declara uma variável do tipo BYTE */
    uint x; /* Declara uma variável do tipo uint */
    unsigned char chmm; /* Os tipos originais podem ser usados */
}
```

Acessando os membros da estrutura

Uma vez criada a variável estrutura, seus membros podem ser acessados por meio do **operador ponto**

- ▶ A instrução

```
Jose.nmat = 456;
```

atribui o valor 456 ao membro **nmat** da variável **Jose**

O *operador ponto* conecta o nome de uma variável estrutura a um membro dela.

exemplos_aulas > structs > C structsA.c > main()

```
1  #include <stdio.h>
2  #include <string.h>
3
4  struct Aluno{
5      char nome[50];
6      int matricula;
7  };
8
9  int main(){
10     struct Aluno aluno1;
11     strcpy(aluno1.nome, "José da Silva");
12     aluno1.matricula = 2024001;
13
14     printf("Dados do aluno1:\n");
15     printf("Nome: %s\n", aluno1.nome);
16     printf("Matrícula: %d\n", aluno1.matricula);
17
18     struct Aluno aluno2 = {"Vanderlério Souza", 2024002};
19
20     printf("Dados do aluno2:\n");
21     printf("Nome: %s\n", aluno2.nome);
22     printf("Matrícula: %d\n", aluno2.matricula);
23
24     return 0;
25 }
```

Definindo uma variável do tipo Aluno, seguido da sua inicialização

Operações entre estruturas

```
exemplos_aulas > structs > C structsC.c > main()
1  #include <stdio.h>
2  #include <string.h>
3
4  typedef struct{
5      int pecas;
6      float preco;
7  } Venda;
8
9  int main(){
10     Venda A = {20, 110.0}, B = {3, 16.5}, Total;
11     //Total = A + B; /* ERRADO */
12     //A soma deve ser efetuada membro a membro
13     Total.pecas = A.pecas + B.pecas;
14     Total.preco = A.preco + B.preco;
15     return 0;
16 }
```

Estruturas aninhadas: composição de estruturas

Exatamente como podemos ter matrizes em que cada elemento é outra matriz, podemos definir estruturas com membros que sejam outras estruturas.


```
/* StructNinho.C */  
/* Mostra estruturas aninhadas */  
#include <stdio.h>  
#include <stdlib.h>
```

```
typedef struct  
{  
    int dia;  
    char mes[10];  
    int ano;  
} Data;
```

```
typedef struct  
{  
    int pecas;  
    float preco;  
    Data diavenda;  
} Venda;
```

```
int main()  
{  
    static Venda A = { 20, 110.0, {7, "Novembro", 2001} };  
  
    printf("Pecas: %d\n", A.pecas);  
    printf("Preco: %.2f\n", A.preco);  
    printf("Data : %d de %s de %d\n", A.diavenda.dia, A.diavenda.mes, A.diavenda.ano);  
  
    printf("Tecle [ENTER] para finalizar ...");  
    getchar();  
    return 0;  
}
```

A estrutura Venda possui um membro do tipo da estrutura Data

Novamente, como em matrizes, a inicialização precisa respeitar o formato da estrutura

Agora temos membros de membros. Daí usamos dois operadores ponto.

Exercitando

1. Crie a estrutura para representar **um elemento** da tabela.

Item	Descrição	Quant.	Unidade	Preço unitário	Obs.	Custo total	Custo Mercadoria Vendida
bas3	Bastidor N°25 (25cm)	1,00	un	16,50		16,50	31,46
ps1	Papel Scrap (média)	3,00	un	3,00		9,00	
POM	Cordão Pompom	0,80	m	0,80		0,64	
PK080	Papel Kraft A4 - 80g	0,50	un	0,10	EMBALAGEM	0,05	
PASB	Papel Sulfite Branco	1,00	un	0,04	p/ DC	0,04	
IMPL	IMPRESSÃO LASER CASEIRA	1,00	un	0,69	p/ DC	0,69	
SA8a	Embalagem de celofane (30x45)	1,00	un	0,25	EMBALAGEM	0,25	
EM9	Caixa p/correspondência papelão C26,5xL19,5xA7,5cm	1,00	un	4,30	EMBALAGEM	4,30	

Exercitando

2. Crie a estrutura para representar a ficha do aluno. Se preciso for, faça composições com outras estruturas.



**FICHA INDIVIDUAL
DO ALUNO**

Nome do aluno: _____

Data de nascimento: ____/____/____

Nome da Mãe: _____

Nome do Pai: _____

Endereço: _____

Telefones para contato: _____

Em caso de emergência avisar: _____

Possui alguma alergia? () sim () não

Se sim, qual? _____

Tem restrição alimentar? _____

Faz uso de algum medicamento: _____

Exercitando

3. Crie a estrutura para representar a ficha do cliente. Se preciso for, faça composições com outras estruturas.

FICHA DE CADASTRO DE CLIENTES		
NOME: 		
CPF: 	RG: 	DATA DE NASC.
ENDEREÇO: 		
CIDADE: 	ESTADO: 	
TELEFONE: 	CELULAR: 	E-MAIL:

Matriz de estrutura

Assim como feito com os tipos nativos de dados, também é possível criar matrizes/vetores de uma estrutura criada.

- ▶ O processo de declaração de uma matriz de estruturas é perfeitamente análogo ao de declaração de qualquer tipo de matriz

- ▶ A instrução

```
Venda vendas[50];
```

declara **vendas** como sendo uma matriz de 50 elementos

- ▶ Cada elemento da matriz é uma estrutura do tipo **Venda**
 - ▶ **vendas[0]** é a primeira estrutura do tipo **Venda**
 - ▶ **vendas[1]** é a segunda estrutura do tipo **Venda**
 - ▶ e assim por diante

Matriz de estrutura

Acessando membros da matriz de estruturas:

- ▶ Membros individuais de cada estrutura são acessados aplicando-se o operador ponto seguido do nome da variável, que, nesse caso, é um elemento da matriz

```
vendas[n].preco
```

- ▶ A expressão acima refere-se ao membro **preco** da n-ésima estrutura da matriz
- ▶ O esquema global desse programa pode ser aplicado a uma grande variedade de situações, como o controle de um estoque
 - ▶ Cada variável poderá conter dados sobre um item do estoque
 - ▶ Número do material
 - ▶ Preço
 - ▶ Tipo
 - ▶ Número de peças do estoque
 - ▶ Etc.

Matriz de estrutura

Exemplo:

```
3
4 struct Data{
5     int dia;
6     char mes[10];
7     int ano;
8 };
9 typedef struct{
10     int pecas;
11     float preco;
12     struct Data diavenda;
13 } Venda;
14 int main(){
15     float total = 0.0;
16     Venda vendas[2];
```

Matriz unidimensional (vetor/array) do tipo Venda, com 2 posições

O uso do tipo descreveu o uso da palavra-chave struct: typedef

```
1 #include <stdio.h>
2
3
4 struct Data{
5     int dia;
6     char mes[10];
7     int ano;
8 };
9 typedef struct{
10     int pecas;
11     float preco;
12     struct Data diavenda;
13 } Venda;
14 int main(){
15     float total = 0.0;
16     Venda vendas[2];
17
18     Venda A = { 20, 110.0, {7,"Fevereiro",2024}};
19     Venda B = { 10, 55.0, {8,"Fevereiro",2024}};
20
21     vendas[0] = A;
22     vendas[1] = B;
23
24     for(int i=0; i < 2; i++){
25         total+= vendas[i].preco;
26     }
27     printf("Total das vendas = R$%.2f\n\n", total);
28     printf("Tecle [ENTER] para finalizar ...");
29     getchar();
30     return 0;
31 }
```

A atribuição de registros no vetor é feito como se faz em um vetor homogêneo.

O acesso ao registro também é feito como se faz em um vetor homogêneo

Agora temos como acessar o membro do elemento do vetor


```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Data{
5      int dia;
6      char mes[10];
7      int ano;
8  };
9  typedef struct{
10     int pecas;
11     float preco;
12     struct Data diavenda;
13 } Venda;
14 int main(){
15     float total = 0.0;
16     Venda vendas[] = {
17         { 20, 110.0, {7,"Fevereiro",2024} },
18         { 10, 55.0, {8,"Fevereiro",2024} }
19     };
20
21     for(int i=0; i < 2; i++){
22         total+= vendas[i].preco;
23     }
24     printf("Total das vendas = R$%.2f\n\n", total);
25     printf("Tecle [ENTER] para finalizar ...");
26     getchar();
27     return 0;
28 }
```

Definição com inicialização

Tipos de dados enumerados: enum

- ▶ Os tipos definidos com **enum**, consistem num conjunto de constantes inteiras, em que cada uma delas é representada por um nome
 - ▶ Estes são chamados de tipos enumerados
- ▶ Tipos enumerados são usados quando conhecemos o conjunto de valores que uma variável pode assumir
- ▶ A variável desse tipo é sempre **int** e, para cada um dos valores do conjunto, atribuímos um nome significativo
- ▶ A palavra **enum** enumera a lista de nomes automaticamente, dando-lhes números em sequência (0, 1, 2, etc.)
- ▶ A vantagem é que utilizamos esses nomes no lugar de números, o que retorna o programa mais claro

Tipos de dados enumerados: enum

- ▶ Vamos escrever um fragmento de programa que define o tipo enumerado e cria uma variável desse tipo para guardar o mês

```
enum Mes {Jan=1, Fev, Mar, Abr, Mai, Jun, Jul, Ago, Set, Out, Nov, Dez};

int main()
{
    enum Mes m1, m2, m3; /* Cria duas variáveis do tipo enum Mes */

    m1 = Abr;             /* Atribui valores */
    m2 = Jun;

    m3 = m2-m1;           /* Operações aritméticas permitidas */
    if(m1 < m2)           /* Comparações permitidas */
}
```

▶ A palavra **enum**

- ▶ Define um conjunto de nomes (cada um deles com um valor inteiro)
- ▶ Enumera esses nome a partir do zero (default) ou, como em nosso programa, a partir do primeiro valor fornecido

Tipos de dados enumerados: enum

- ▶ Vamos escrever um fragmento de programa que define o tipo enumerado e cria uma variável desse tipo para guardar o mês

Assim como struct!

```
enum Mes {Jan=1, Fev, Mar, Abr, Mai, Jun, Jul, Ago, Set, Out, Nov, Dez};
```

```
int main()
```

```
{
```

```
    enum Mes m1, m2, m3; /* Cria duas variáveis do tipo enum Mes */
```

```
    m1 = Abr; /* Atribui valores */
```

```
    m2 = Jun;
```

```
    m3 = m2 - m1; /* Operações aritméticas permitidas */
```

```
    if(m1 < m2) /* Comparações permitidas */
```

O valor é 4

▶ A palavra **enum**

- ▶ Define um conjunto de nomes (cada um deles com um valor inteiro)
- ▶ Enumera esses nome a partir do zero (default) ou, como em nosso programa, a partir do primeiro valor fornecido

Tipos de dados enumerados: enum

- ▶ Vamos escrever um fragmento de programa que define o tipo enumerado e cria uma variável desse tipo para guardar o mês

```
enum Mes {Jan=1, Fev, Mar, Abr, Mai, Jun, Jul, Ago, Set, Out, Nov, Dez};
```

```
int main()  
{
```

```
    enum Mes m1, m2, m3; /* Cria duas variáveis do tipo enum Mes */
```

```
    m1 = sabado; /* ERRADO. É ilegal */
```

```
    m2 = Jun;
```

```
    m3 = m2-m1; /* Operações aritméticas permitidas */
```

```
    if(m1 < m2) /* Comparações permitidas */
```

Valor não existe na lista

▶ A palavra **enum**

- ▶ Define um conjunto de nomes (cada um deles com um valor inteiro)
- ▶ Enumera esses nome a partir do zero (default) ou, como em nosso programa, a partir do primeiro valor fornecido

Tipos de dados enumerados: enum

Usando typedef com enum:

Agora temos um apelido

```
typedef enum {Jan=1, Feb, Mar, Abr, Mai, Jun, Jul, Ago, Set, Out, Nov, Dez} Mes;  
  
int main()  
{  
    Mes m1, m2, m3;      /* Agora o nome do tipo é Mês          */  
    m1 = Abr;             /* Atribui valores          */  
    m2 = Jun;  
    m3 = m2-m1;           /* Operações aritméticas permitidas */  
    if(m1 < m2)           /* Comparações permitidas    */  
}
```

Não há uso da palavra-chave enum

```
exemplos_aulas > structs > C enumB.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef enum {FALSO, VERDADEIRO} Boleano;
5
6  int main(){
7      Boleano bool = FALSO;
8
9      if (bool)
10         printf("O valor é verdadeiro\n");
11     else
12         printf("O valor é falso\n");
13
14     printf("Tecle [ENTER] para finalizar ...");
15     getchar();
16     return 0;
17 }
```

Vai entrar no bloco
verdade ou no senão?

Exercitando

4. Considerando as estruturas abaixo, crie uma estrutura para representar o curso de Bacharelado em Eng. de Software do IFG Inhumas.

Disciplina	
Nome	texto
Codigo	número
C.H.	número
Semestre	enum
Prof. resp	Professor

Professor	
Nome	texto
Matricula	número
e-mail	texto
Genero	enum

Curso	
Nome	texto
Total semestres	número
C.H. total	número
Turno	enum
Coordenador	Professor
Disciplinas	Disciplina
Professores	Professor